

vLLM专题（五）-生成模型



大模型专题系列 专栏收录该内容

¥49.90
¥99.00

111 篇文章

已订阅

8折续

👑 您已是超级会员，正在免费阅读会员专享内容

[查看更多超级会员权益](#) >

vLLM 为生成模型提供了优质的支持，涵盖了大部分的大型语言模型（LLM）。

在 vLLM 中，生成模型实现了 `VllmModelForTextGeneration` 接口。基于输入的最终隐状态，这些模型输出生成标记的对数概率，然后通过采样器（Sampler）获得最终的文本。

对于生成模型，唯一支持的 `--task` 选项是“generate”。通常，这会自动推断，因此您不需要显式指定。

1. 离线推理

LLM 类提供了多种方法用于离线推理。有关初始化模型时的选项，请参阅引擎参数（Engine Arguments）。

1.1 LLM.generate

`generate` 方法适用于 vLLM 中的所有生成模型。它类似于 HF Transformers 中的相应方法，不同之处在于，标记化和去标记化过程也会自动执行。

```
llm = LLM(model="facebook/opt-125m")
outputs = llm.generate("Hello, my name is")

for output in outputs:
    prompt = output.prompt
    generated_text = output.outputs[0].text
    print(f"Prompt: {
        prompt!r}, Generated text: {
        generated_text!r}")
```

您可以通过传递 `SamplingParams` 来可选地控制语言生成。例如，您可以通过设置 `temperature=0` 来使用贪婪采样（greedy sampling）：

```
llm = LLM(model="facebook/opt-125m")
params = SamplingParams(temperature=0)
outputs = llm.generate("Hello, my name is", params)

for output in outputs:
    prompt = output.prompt
    generated_text = output.outputs[0].text
    print(f"Prompt: {
        prompt!r}, Generated text: {
        generated_text!r}")
```

代码示例可以在这里找到: [examples/offline_inference/basic.py](https://github.com/facebookresearch/llm/blob/main/examples/offline_inference/basic.py)

1.2 LLM.beam_search

`beam_search` 方法在 `generate` 的基础上实现了束搜索 (beam search)。例如, 要使用 5 个束进行搜索并最多输出 50 个标记:

```
llm = LLM(model="facebook/opt-125m")
params = BeamSearchParams(beam_width=5, max_tokens=50)
outputs = llm.generate("Hello, my name is", params)

for output in outputs:
    prompt = output.prompt
    generated_text = output.outputs[0].text
    print(f"Prompt: {
        prompt!r}, Generated text: {
        generated_text!r}")
```

1.3 LLM.chat

`chat` 方法在 `generate` 的基础上实现了聊天功能。特别地, 它接受类似于 OpenAI Chat Completions API 的输入, 并自动应用模型的聊天模板来格式化提示。

重要提示

通常, 只有经过指令调优的模型才具有聊天模板。基础模型可能表现较差, 因为它们并未经过训练来响应聊天对话。

```

llm = LLM(model="meta-llama/Meta-Llama-3-8B-Instruct")
conversation = [
    {
        "role": "system",
        "content": "You are a helpful assistant"
    },
    {
        "role": "user",
        "content": "Hello"
    },
    {
        "role": "assistant",
        "content": "Hello! How can I assist you today?"
    },
    {
        "role": "user",
        "content": "Write an essay about the importance of higher education.",
    },
]
outputs = llm.chat(conversation)

for output in outputs:
    prompt = output.prompt
    generated_text = output.outputs[0].text
    print(f"Prompt: {
        prompt!r}, Generated text: {
        generated_text!r}")

```

代码示例可以在这里找到：[examples/offline_inference/chat.py](#)

如果模型没有聊天模板，或者您想指定另一个模板，您可以显式地传递一个聊天模板：

```

from vllm.entrypoints.chat_utils import load_chat_template

# You can find a list of existing chat templates under `examples/`
custom_template = load_chat_template(chat_template="<path_to_template>")
print("Loaded chat template:", custom_template)

outputs = llm.chat(conversation, chat_template=custom_template)

```

2. 在线服务

我们的OpenAI兼容服务器提供了与离线API相对应的端点：

- Completions API 类似于 LLM.generate，但只接受文本。
- Chat API 类似于 LLM.chat，接受文本和多模态输入，适用于具有聊天模板的模型。